Modelos de regresión en Python

Pipelines, evaluación y búsqueda de hiperparámetros

Material didáctico · IFCD107

21 de octubre de 2026

Resumen

Este documento explica la organización y el funcionamiento de varios métodos de regresión implementados con Python y scikit-learn: regresión lineal, K vecinos, regresión polinómica, bosques aleatorios, SVR, AdaBoost y Gradient Boosting. Describe el preprocesamiento tabular, la separación entre entrenamiento y prueba, la validación cruzada, las métricas y las búsquedas `RandomizedSearchCV` y `GridSearchCV`. El objetivo es conectar cada bloque de código con las decisiones estadísticas y de ingeniería que lo motivan.

Índice

1. Objetivos y mapa del proyecto **3**

2. Flujo de aprendizaje supervisado **3**
2.1. Separación entrenamiento/prueba . . . 3
2.2. Validación cruzada . . . 3

3. Preprocesamiento tabular y pipelines **4**

4. Algoritmos de regresión **4**
4.1. Regresión lineal . . . 4
4.2. K vecinos más próximos . . . 5
4.3. Regresión polinómica . . . 5
4.4. Bosque aleatorio . . . 5
4.5. SVR . . . 5
4.6. AdaBoost . . . 6
4.7. Gradient Boosting . . . 6

5. Métricas para interpretar el rendimiento **6**

6. Búsqueda de hiperparámetros **6**
6.1. `RandomizedSearchCV` . . . 7
6.2. `GridSearchCV` . . . 7

7. Artefactos, API y operación **7**

8. Ejemplo de selección responsable **7**

9. Catálogo de parámetros: significado, valores e interpretación **8**
9.1. Preprocesadores y composición . . . 8
9.2. Regresión lineal y K vecinos . . . 10
9.3. Regresión polinómica con NumPy . . . 11
9.4. Bosque aleatorio . . . 12

9.5. SVR 13
9.6. AdaBoost para regresión 13
9.7. Gradient Boosting para regresión 14
9.8. Particiones, validación y búsqueda 15
9.9. Métricas e interpretación de scores 16
9.10. Parámetros del wrapper, datos y valores de referencia 17

10. Conclusiones **18**

1. Objetivos y mapa del proyecto

Una tarea de regresión estima una variable numérica continua a partir de una o más variables de entrada. El proyecto organiza el flujo en módulos para que la API, el preprocesamiento, los estimadores y la evaluación tengan responsabilidades claras.

`app.py` — expone las rutas HTTP y recibe las solicitudes.

`api_functions.py` — valida entradas, coordina el entrenamiento, selecciona la implementación y reúne resultados y artefactos.

`regression/tabular_preprocessing.py` — construye transformaciones reutilizables para columnas numéricas y categóricas.

`regression/linear_regressor.py`, **`knn_regressor.py`**, **`polynomial_regressor.py`** — contienen estimadores y pipelines específicos.

`regression/random_forest_regressor.py`, **`svm_regressor.py`**, **`adaboost_regressor.py`**, **`gradient_boosting_regressor.py`** — implementan los demás algoritmos.

La API actúa como coordinadora: prepara los datos, invoca el módulo elegido y devuelve métricas, predicciones y, cuando corresponde, ficheros de salida. Mantener las clases y funciones del estimador fuera de las rutas HTTP facilita reutilizarlas desde otros servicios o tareas por lotes.

2. Flujo de aprendizaje supervisado

El ciclo habitual consta de: (1) validar columnas y tipos; (2) separar los datos en entrenamiento y prueba; (3) ajustar el preprocesamiento y el modelo con entrenamiento; (4) elegir hiperparámetros usando solo entrenamiento; (5) medir una sola vez sobre el conjunto de prueba; y (6) guardar el modelo final y sus resultados. La variable objetivo y no debe aparecer entre las entradas X.

2.1. Separación entrenamiento/prueba

`train_test_split` reserva observaciones para estimar el rendimiento en datos no usados al ajustar. Una configuración común es `test_size=0.2` y `random_state=42`. La semilla hace reproducible la partición, aunque no convierte el resultado en universal: otro conjunto de datos puede producir métricas distintas.

La prueba debe permanecer al margen de la selección de hiperparámetros. Si se usa para elegir el modelo y luego se informa su puntuación como evaluación final, la estimación queda optimistamente sesgada.

2.2. Validación cruzada

En validación `KFold`, se divide el conjunto de entrenamiento en k partes. En cada iteración se entrena con $k-1$ partes y se valida con la restante; las puntuaciones se resumen entre iteraciones. Con $k = 5$, cada observación participa en validación una vez. `LeaveOneOut` es el caso extremo en que se deja fuera una observación cada vez: puede ser útil en muestras pequeñas, pero implica muchos ajustes.

`cross_val_predict` genera predicciones fuera de muestra para cada fila: la predicción de una fila procede de un modelo que no la vio durante su ajuste. Estas predicciones permiten calcular métricas agregadas sin confundirlas con las predicciones de entrenamiento.

3. Preprocesamiento tabular y pipelines

Los datos reales mezclan números, categorías y valores ausentes. `ColumnTransformer` aplica transformaciones distintas por columna; `Pipeline` enlaza esas transformaciones con el estimador.

Cuadro 1: Transformaciones habituales del preprocesador tabular.

Tipo de dato	Transformación	Motivo
Numérico	`SimpleImputer(strategy="median")`	Completa valores ausentes con una estadística resistente a valores extremos.
Categórico	imputación por moda y `OneHotEncoder(handle_unknown="ignore")`	Convierte categorías a columnas numéricas y tolera categorías nuevas en inferencia.
Numérico escalado	`StandardScaler` opcional	Centra y escala características cuando el algoritmo depende de distancias o magnitudes.

El escalado es especialmente relevante para KNN y SVR. Un atributo medido en miles podría dominar a otro medido entre cero y uno si se usan distancias o márgenes sin normalización. Los árboles y los bosques suelen ser menos sensibles a la escala.

El pipeline debe ajustarse dentro de cada pliegue de validación. Así, medianas, modas, categorías y escalas se calculan solo con el subconjunto de entrenamiento de esa iteración. Ajustar el preprocesador una vez antes de validar filtra información del futuro conjunto de validación.

Listing 1: Esquema de un pipeline tabular

```
preprocessor = build_tabular_preprocessor(
    numeric_features=numeric_columns,
    categorical_features=categorical_columns,
    scale_numeric=True,
)
model = Pipeline([
    ("preprocessor", preprocessor),
    ("regressor", SVR(kernel="rbf", C=1.0, epsilon=0.1)),
])
model.fit(X_train, y_train)
predictions = model.predict(X_test)
```

4. Algoritmos de regresión

4.1. Regresión lineal

`LinearRegression` ajusta una relación lineal entre predictores y objetivo, minimizando la suma de errores cuadrados. Para un predictor, la forma es $\hat{y} = \beta_0 + \beta_1 x$. El módulo de regresión lineal valida que los valores sean finitos y que haya suficientes filas, reserva una partición de prueba y calcula las métricas. Después puede volver a ajustar un modelo de servicio con todas las observaciones disponibles. Ese modelo completo sirve para inferencia; las métricas corresponden a la evaluación previa con el conjunto reservado.

La regresión lineal es un punto de referencia sencillo e interpretable. Una relación curva, interacciones ausentes o valores atípicos pueden limitar su rendimiento.

4.2. K vecinos más próximos

`KNeighborsRegressor` estima una respuesta combinando los objetivos de los vecinos más cercanos. El pipeline `build_knn_pipeline` escala las variables antes de medir distancias y utiliza cinco vecinos como valor de referencia. Un k pequeño puede seguir el ruido; uno grande suaviza más y puede perder estructura local.

La evaluación puede usar `KFold` o `LeaveOneOut`, y las predicciones fuera de pliegue permiten calcular errores de validación. Finalmente, el pipeline seleccionado se ajusta con todos los datos de entrenamiento disponibles.

4.3. Regresión polinómica

La regresión polinómica amplía la base de características con potencias de x:

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2 + \cdots + \beta_d x^d.$$

El grado d controla la flexibilidad. Un grado bajo puede no captar la curvatura; uno alto puede ajustarse demasiado al ruido y comportarse mal fuera del rango observado.

En la implementación, `PolynomialRegressor` utiliza `numpy.polyfit` y `numpy.polyval` para una variable predictora. Valida el grado (entre 1 y 10), la finitud de los datos y la cantidad y diversidad de valores de entrada. La opción de pesos modifica la importancia relativa de las observaciones; `rcond`, `full` y `cov` controlan opciones del ajuste numérico.

`PolynomialSearchEstimator` adapta el estimador a la interfaz de scikit-learn. `build_polynomial_search` construye un pipeline y una búsqueda aleatoria. La cuadrícula del proyecto contempla grados 1 a 5, valores de `rcond`, `full`, pesos y covarianza (80 combinaciones nominales); la configuración activa explora 20 candidatos, con cinco pliegues, semilla 42, puntuación `neg_mean_squared_error` y reajuste del mejor candidato. La API separa una prueba final del 20 % y realiza la búsqueda únicamente dentro del conjunto de entrenamiento.

Variante con Ollama. En la copia autorizada del proyecto IFCD107 existe `PolynomialSearchEstimatorOllar` que puede seleccionarse mediante `estimator_class`. Ollama redacta una explicación de los resultados finales después de la búsqueda. La generación de texto no participa en el ajuste de cada pliegue ni sustituye las métricas numéricas: es una capa de interpretación. La disponibilidad y el modelo de Ollama dependen de que el servicio esté desplegado y accesible.

4.4. Bosque aleatorio

`RandomForestRegressor` promedia árboles entrenados con muestras y subconjuntos de características aleatorizados. El módulo usa como base 200 árboles, profundidad sin límite explícito, paralelismo con `n_jobs=-1` y semilla 42. El preprocesamiento imputa valores y codifica categorías. La importancia de características puede ayudar a explorar qué variables contribuyeron a las divisiones, pero no demuestra causalidad y puede favorecer atributos con muchas posibilidades de corte.

4.5. SVR

La regresión por vectores de soporte (`SVR`) busca una función que mantenga errores dentro de una banda controlada por `epsilon`, equilibrando ajuste y complejidad mediante `C`. Admite kernels `linear`, `poly`, `rbf` y `sigmoid`; `gamma` controla el alcance de los kernels no lineales y `degree` el grado del kernel polinómico. El preprocesamiento con escalado es importante porque la geometría de los datos determina el margen y las distancias.

4.6. AdaBoost

AdaBoost ensambla sucesivamente estimadores débiles. El módulo utiliza árboles de decisión poco profundos como estimadores base; los ejemplos difíciles influyen más en las etapas siguientes. `n_estimators`, `learning_rate` y la pérdida controlan tamaño y ritmo del conjunto. Si cada etapa aprende demasiado, el modelo puede sobreajustar; si aprende poco, puede quedarse corto.

4.7. Gradient Boosting

`GradientBoostingRegressor` construye árboles secuenciales para corregir gradualmente los errores residuales. La profundidad controla la complejidad de cada árbol, mientras que `learning_rate` y `n_estimators` se equilibran entre sí. Se admiten pérdidas como `squared_error`, `absolute_error`, `huber` y `quantile`; `subsample` puede introducir aleatoriedad en el ajuste.

5. Métricas para interpretar el rendimiento

Sean y_i los valores reales, $\hat{y}_i$ las predicciones y n el número de observaciones evaluadas.

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|,$$

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2},$$

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}.$$

- **MAE**: error absoluto medio en las unidades del objetivo. Se interpreta como el tamaño medio del error y es menos sensible a errores extremos que RMSE.
- **RMSE**: también está en las unidades del objetivo, pero penaliza más los errores grandes por elevarlos al cuadrado.
- R^2: compara el error del modelo con una predicción constante basada en la media. Puede ser negativo en prueba cuando el modelo rinde peor que esa referencia.
- **Sesgo**: la media de $y_i - \hat{y}_i$ indica la dirección media del error según esta convención; un signo positivo significa que el modelo tiende a quedarse corto.
- **Correlación de Pearson**: resume asociación lineal entre dos series; no mide por sí sola el tamaño del error ni reemplaza MAE, RMSE o R^2.

Las métricas deben calcularse sobre datos no usados para ajustar el modelo y deben interpretarse según el coste real de equivocarse.

6. Búsqueda de hiperparámetros

Los hiperparámetros se fijan antes de ajustar: por ejemplo, grado polinómico, número de vecinos o profundidad de árbol. Para cada configuración se entrena y valida el pipeline, lo cual permite comparar opciones con un procedimiento común.

6.1. `RandomizedSearchCV`

La búsqueda aleatoria recibe una lista de combinaciones o distribuciones y selecciona un número acotado mediante `n_iter`. Es útil cuando el espacio es amplio o cuando el presupuesto de cómputo limita una exploración exhaustiva. En el ajuste polinómico descrito, se prueban 20 candidatos y cinco pliegues; en total son hasta 100 evaluaciones de pliegue, más el reajuste final del mejor candidato.

6.2. `GridSearchCV`

La búsqueda en cuadrícula evalúa todas las combinaciones especificadas en `param_grid`. Si se recorrieran las 80 combinaciones polinómicas nominales con cinco pliegues, serían hasta 400 evaluaciones de pliegue, más el reajuste. `GridSearchCV` no está importado ni utilizado en el código descrito; esta cifra ilustra cómo se configuraría una comparación exhaustiva con esa cuadrícula. El total real puede reducirse si se usa una lista de parámetros que excluya combinaciones inválidas o redundantes.

Cuadro 2: Comparación conceptual de búsquedas.

Método	Evaluación	Cuándo conviene
`RandomizedSearchCV`	Muestra hasta `n_iter` candidatos; el coste es configurable.	Espacios grandes, distribuciones o presupuesto limitado.
`GridSearchCV`	Recorre todas las combinaciones declaradas.	Espacios pequeños donde interesa una comparación exhaustiva y reproducible.

En ambos casos debe usarse una métrica coherente con el problema y reservarse el conjunto de prueba para el informe final. La mejor puntuación de validación no garantiza que el modelo sea el mejor para cualquier población futura.

7. Artefactos, API y operación

El proyecto organiza los ficheros producidos en ubicaciones como `csv_entrada`, `csv_salida`, `graficas`, `modelos` y `logs_salida`. La API recibe datos JSON o CSV, comprueba estructura y objetivo, selecciona un flujo de regresión y devuelve un identificador de ejecución con resultados. Conservar entradas, métricas y configuración ayuda a reproducir una ejecución.

Los modelos serializados, por ejemplo en formatos basados en pickle, deben tratarse como código ejecutable: solo deben cargarse artefactos de confianza y se deben registrar versiones de Python y bibliotecas. Para inferencia, las columnas y transformaciones deben coincidir con las usadas durante el entrenamiento; guardar el pipeline completo reduce el riesgo de aplicar pasos distintos.

8. Ejemplo de selección responsable

Un análisis práctico puede comparar una regresión lineal como referencia con KNN, SVR, bosque aleatorio y boosting. Para cada opción:

1. Definir las columnas predictoras y el objetivo; revisar unidades, valores ausentes y posibles fugas.

2. Mantener una separación de prueba reproducible.

3. Construir un pipeline que incluya imputación, codificación y escalado cuando proceda.

4. Ajustar o buscar hiperparámetros solo con entrenamiento, usando validación cruzada.

5. Comparar MAE, RMSE y R^2 en la prueba y revisar residuos.

6. Elegir considerando precisión, coste de inferencia, interpretabilidad y mantenimiento.

7. Guardar el pipeline y metadatos suficientes para reconstruir la ejecución.

No existe un algoritmo ganador para todos los conjuntos de datos. El resultado depende de la distribución, el tamaño de muestra, la calidad de las variables y el objetivo de uso.

9. Catálogo de parámetros: significado, valores e interpretación

Esta sección documenta los parámetros públicos de los estimadores y utilidades citados en el material, contrastándolos con la documentación oficial de scikit-learn y NumPy. Los nombres de parámetros se conservan en inglés porque así aparecen en Python; sus explicaciones están en castellano. Los valores por omisión pueden cambiar entre versiones, por lo que se debe consultar la versión instalada antes de reproducir un experimento. La documentación en línea consultada corresponde a la rama estable disponible al redactar este material.

Hay dos niveles distintos. Los parámetros de los estimadores de scikit-learn son los del constructor de cada clase. Los builders del proyecto (`build_knn_pipeline`, `build_tabular_preprocessor`, etc.) pueden exponer solo una parte de ellos y fijar el resto internamente. Por ejemplo, que `RandomForestRegressor` admita `max_features` no significa que la función de alto nivel permita cambiarlo. Para la lista efectiva de argumentos del proyecto hay que leer la firma de cada builder.

9.1. Preprocesadores y composición

`Pipeline`.

- **`steps`** — Lista ordenada de pares (nombre, transformador/estimador); cada etapa recibe la salida de la anterior y la última suele ser el modelo.
- **`memory=None`** — Desactiva la caché; una ruta o `joblib.Memory` permite reutilizar transformaciones costosas.
- **`verbose=False`** — Oculta los tiempos de las etapas; con `True` los imprime.

Los parámetros internos usan nombres anidados como `regressor__C`. El pipeline debe ajustarse dentro de cada pliegue para evitar fuga. Referencia: https://scikit-learn.org/stable/modules/generated/sklearn.pipeline.Pipeline.html.

`ColumnTransformer`.

- **`transformers`** — Lista de tuplas (nombre, transformador, selector de columnas); los selectores pueden ser nombres, índices, máscaras o selectores de columnas.
- **`remainder='drop'`** — Descarta columnas no mencionadas; acepta también `'passthrough'` o un transformador para tratarlas.
- **`sparse_threshold=0.3`** — Umbral de densidad entre 0 y 1 que decide si la salida combinada será dispersa; 0 fuerza salida densa.
- **`n_jobs=None`** — Usa el paralelismo por defecto; `-1` solicita todos los núcleos.
- **`transformer_weights=None`** — No pondera ramas; un diccionario multiplica la salida de cada rama por su peso.

- **verbose=False** — No imprime tiempos de ajuste; verdadero muestra el tiempo por transformador.
- **verbose_feature_names_out=True** — Prefija los nombres de salida con el nombre del transformador; versiones recientes admiten formato o callable.

SimpleImputer.

- **missing_values=np.nan** — Valor que identifica una ausencia; según el tipo, también se admite `pd.NA`.
- **strategy='mean'** — Estrategia de sustitución: `'mean'`, `'median'`, `'most_frequent'`, `'constant'` o callable en versiones recientes. La media requiere números; la moda sirve para categorías.
- **fill_value=None** — Valor utilizado cuando la estrategia es constante; por defecto suele ser 0 para números y `'missing_value'` para cadenas.
- **copy=True** — Conserva la entrada; falso intenta ahorrar memoria y puede modificarla según formato y tipo.
- **add_indicator=False** — No agrega indicadores; verdadero añade columnas binarias que señalan las ausencias vistas durante el ajuste.
- **keep_empty_features=False** — Permite descartar columnas vacías en entrenamiento; verdadero las conserva.

El preprocesador del proyecto utiliza mediana para números y moda para categorías.

OneHotEncoder.

- **categories='auto'** — Descubre los valores observados; una lista de listas fija las categorías y su orden.
- **drop=None** — Conserva todas las columnas; admite `'first'`, `'if_binary'` o categorías explícitas para descartar una columna.
- **sparse_output=True** — Devuelve matriz CSR dispersa; falso produce un array denso. Antes de scikit-learn 1.2 el argumento se llamaba `sparse`.
- **dtype=np.float64** — Tipo numérico de la salida.
- **handle_unknown='error'** — Rechaza categorías nuevas; `'ignore'` las codifica como ceros; `'infrequent_if_exist'` las asigna al grupo infrecuente si existe; `'warn'` avisa y aplica ese último criterio en versiones compatibles.
- **min_frequency=None** — No agrupa categorías raras; un entero expresa frecuencia absoluta y un decimal, proporción mínima.
- **max_categories=None** — No limita columnas de salida; un entero fija el máximo por predictor, incluida la categoría infrecuente.
- **feature_name_combiner='concat'** — Combina nombre de predictor y categoría; un callable permite definir otro formato.

El proyecto fija `handle_unknown='ignore'` para admitir categorías nuevas en inferencia.

`StandardScaler`.

- `copy=True` — Conserva la entrada cuando es posible; falso intenta operar en el mismo almacenamiento.
- `with_mean=True` — Resta la media; debe ser falso para matrices dispersas, porque centrarlas las densifica.
- `with_std=True` — Divide por la desviación típica; falso conserva la escala de varianza.

Estandarizar no garantiza normalidad ni limita valores: solo usa estadísticas del conjunto de ajuste. El builder puede aplicar el escalado solo a columnas numéricas.

9.2. Regresión lineal y K vecinos

`LinearRegression`.

- `fit_intercept=True` — Estima el término independiente; falso presupone datos centrados o una recta que pasa por el origen.
- `copy_X=True` — Protege la matriz de entrada; falso permite sobrescribirla en ciertos cálculos.
- `tol=1e-6` — Tolerancia del solver; su disponibilidad e interpretación dependen de versión y de matriz densa o dispersa.
- `n_jobs=None` — Deja el paralelismo por defecto; solo acelera casos particulares multisalida con matriz dispersa o coeficientes positivos; `-1` solicita todos los procesadores.
- `positive=False` — Permite coeficientes de cualquier signo; verdadero impone coeficientes no negativos y requiere matriz densa.
- `sample_weight=None en fit` — No es argumento del constructor; admite un peso por observación para ponderar el ajuste.

El coeficiente se interpreta como cambio esperado en y por unidad del predictor, manteniendo los demás constantes bajo el supuesto lineal. El wrapper del proyecto puede restringir la entrada a un único predictor.

`KNeighborsRegressor`.

- `n_neighbors=5` — Número de vecinos; entero positivo no mayor que las filas de entrenamiento de cada pliegue. Un valor pequeño sigue variaciones locales; uno grande suaviza más.
- `weights='uniform'` — Pondera por igual; `'distance'` da más peso a vecinos cercanos; callable permite una regla propia.
- `algorithm='auto'` — Elige entre `ball_tree`, `kd_tree` y `brute`; afecta principalmente el coste de consulta.
- `leaf_size=30` — Afecta memoria y velocidad de los árboles espaciales, pero normalmente no las predicciones.
- `p=2` — Parámetro de Minkowski: 1 corresponde a distancia Manhattan; 2, euclídea; otros valores positivos definen otras normas.
- `metric='minkowski'` — Métrica de distancia; acepta otra métrica válida de SciPy o una callable.
- `metric_params=None` — Opciones específicas para la métrica elegida.
- `n_jobs=None` — Paralelismo durante consultas; `-1` solicita todos los núcleos.

El escalado previo cambia la noción de cercanía y forma parte esencial del pipeline.

9.3. Regresión polinómica con NumPy

`numpy.polyfit`.

- **`x`** — Valores del predictor; debe compartir longitud con y y normalmente ser finito.
- **`y`** — Variable objetivo; debe tener el mismo número de observaciones que x.
- **`deg`** — Grado entero no negativo; determina la potencia máxima. Un grado alto aumenta flexibilidad y riesgo de oscilación/sobreajuste.
- **`rcond=None`** — Umbral relativo basado por defecto en longitud de x y precisión flotante; descarta valores singulares menores. Reducirlo puede conservar información inestable.
- **`full=False`** — Devuelve coeficientes; verdadero añade residuos, rango, valores singulares y umbral empleado.
- **`w=None`** — Sin ponderación; un vector positivo pondera el residuo sin elevarlo al cuadrado. Para desviación conocida puede usarse $w_i = 1/\sigma_i$.
- **`cov=False`** — No devuelve covarianza; verdadero devuelve una estimación escalada por chi-cuadrado reducido; **`'unscaled'`** omite esa escala si las incertidumbres absolutas están calibradas.

La forma de salida cambia con `full=True`, y NumPy establece restricciones al combinar diagnósticos y covarianza. Los coeficientes salen de mayor a menor potencia. `polyfit` es la API antigua; NumPy recomienda `numpy.polynomial.Polynomial.fit` por estabilidad numérica.

`PolynomialRegressor` del proyecto.

- **`degree`** — Grado del polinomio; el wrapper limita su intervalo a 1–10 y valida suficientes observaciones y valores distintos de x.
- **`rcond`** — Umbral numérico transferido a `numpy.polyfit`; controla qué valores singulares se descartan.
- **`full`** — Solicita diagnósticos del ajuste numérico además de los coeficientes.
- **`w`** — Pesos por observación; una lista de pesos fija no es adecuada para pliegues de tamaños distintos.
- **`cov`** — Controla si se calcula covarianza de coeficientes y cómo se escala.
- **`x, y`** — Vectores de entrada y objetivo con la misma longitud y valores finitos.

El wrapper `PolynomialRegressor` del curso limita y valida el grado (1–10), controla valores finitos, cantidad de muestras y valores de x distintos, y ofrece opciones como pesos y diagnóstico según su firma. No hereda automáticamente todo el API de sklearn. En el espacio de búsqueda documentado:

- **`degree`** — Se prueban grados de 1 a 5, dentro del intervalo permitido por el wrapper.
- **`rcond`** — Se prueban `None`, `1e-12`, `1e-10` y `1e-8`; el valor controla el umbral numérico.
- **`full`** — Se prueban `False` y `True`; activa o desactiva los diagnósticos del ajuste.
- **`w`** — Se fija en `None`; no se ponderan observaciones.
- **`cov`** — Se prueban `False` y `'unscaled'`; modifica el cálculo y retorno de la covarianza.

No todas las combinaciones son semánticamente independientes: el grado y el umbral afectan el ajuste, mientras que `full` solicita diagnósticos y `cov` cambia el retorno.

9.4. Bosque aleatorio

`RandomForestRegressor.`

- `n_estimators=100` — Cantidad de árboles; más árboles aumentan tiempo y memoria y suelen estabilizar el promedio.
- `criterion='squared_error'` — Criterio de división: también `'absolute_error'`, `'friedman_mse'` o `'poisson'`; cada uno define una pérdida adecuada a distintos objetivos.
- `max_depth=None` — Profundidad máxima; None deja crecer hasta otros límites; entero reduce complejidad.
- `min_samples_split=2` — Mínimo para dividir un nodo; acepta entero o fracción de filas.
- `min_samples_leaf=1` — Mínimo por hoja; entero o fracción. Elevarlo suaviza el modelo.
- `min_weight_fraction_leaf=0` — Fracción mínima del peso total requerida en cada hoja.
- `max_features=1.0` — Predictores considerados por división; admite entero, fracción, `'sqrt'`, `'log2'` o None, según versión. Reducirlo añade aleatoriedad.
- `max_leaf_nodes=None` — Sin límite de hojas; un entero limita nodos y prioriza las divisiones con mayor mejora.
- `min_impurity_decrease=0` — Mejora mínima de impureza para aceptar una división.
- `bootstrap=True` — Muestrea filas con reemplazo; falso usa todas las filas para cada árbol.
- `oob_score=False` — No calcula puntuación out-of-bag; verdadero la calcula si bootstrap está activo; admite callable en versiones recientes.
- `n_jobs=None` — Paralelismo; `-1` solicita todos los núcleos.
- `random_state=None` — Sin semilla fija; un entero hace reproducible el muestreo.
- `verbose=0` — Nivel de mensajes durante el ajuste.
- `warm_start=False` — Reajusta desde cero; verdadero conserva árboles y añade hasta el nuevo `n_estimators`.
- `ccp_alpha=0` — Sin poda por complejidad; valores mayores podan ramas.
- `max_samples=None` — Con bootstrap usa todas las filas por árbol; admite entero o fracción.
- `monotonic_cst=None` — Sin restricciones monótonas; vector de valores $-1, 0, 1$ restringe el sentido del efecto por predictor, sujeto a límites de compatibilidad de sklearn.

En el wrapper del curso la configuración de referencia es 200 árboles, profundidad ilimitada, todos los trabajos y semilla 42; los demás parámetros suelen conservar defaults de sklearn. La importancia basada en reducción de impureza sirve como orientación, no como causalidad, y puede favorecer variables categóricas de alta cardinalidad.

9.5. SVR

`SVR.`

- `kernel='rbf'` — Función de similitud; admite `'linear'`, `'poly'`, `'rbf'`, `'sigmoid'` o callable.
- `degree=3` — Grado del kernel polinómico; no afecta a los demás kernels.
- `gamma='scale'` — Usa $1/(n_features\,Var(X))$; `'auto'` usa $1/n_features$; también admite número positivo para controlar el alcance del kernel.
- `coef0=0` — Desplazamiento que afecta a kernels polinómico y sigmoide.
- `tol=0.001` — Tolerancia de parada; reducirla puede aumentar precisión y tiempo de ajuste.
- `C=1` — Penalización de errores; valores altos dan más importancia al ajuste de entrenamiento y pueden sobreajustar.
- `epsilon=0.1` — Anchura, en unidades del objetivo, de la banda de error no penalizada; mayor suele simplificar el modelo.
- `shrinking=True` — Activa una heurística que suele acelerar el ajuste; se puede desactivar para diagnosticar convergencia.
- `cache_size=200` — Memoria de caché en MB.
- `verbose=False` — Oculta mensajes de avance; verdadero los muestra.
- `max_iter=-1` — Sin límite de iteraciones; un entero positivo establece un tope.

SVR puede escalar peor que linealmente en cantidad de muestras; escalar predictores es esencial y también puede convenir escalar el objetivo.

9.6. AdaBoost para regresión

`AdaBoostRegressor.`

- `estimator=None` — Estimador base; el default de sklearn es un árbol pequeño y puede cambiar por versión. Debe aceptar pesos de muestra. El builder del proyecto usa un árbol de profundidad 2.
- `n_estimators=50` — Máximo de etapas; puede detenerse antes si alcanza error perfecto.
- `learning_rate=1` — Peso de cada etapa; una tasa menor suele requerir más estimadores.
- `loss='linear'` — Regla de actualización de pesos; también admite `'square'` y `'exponential'`, que penalizan de manera distinta los errores grandes.
- `random_state=None` — Sin semilla fija; un entero hace reproducible el ajuste.

El builder del proyecto usa normalmente 100 etapas y tasa 0.05, además de exponer la pérdida y profundidad del árbol base.

9.7. Gradient Boosting para regresión

`GradientBoostingRegressor.`

- `loss='squared_error'` — Pérdida cuadrática; alternativas `'absolute_error'`, `'huber'` (robusta) y `'quantile'`.
- `learning_rate=0.1` — Reduce la contribución de cada árbol; se equilibra con la cantidad de etapas.
- `n_estimators=100` — Número máximo de etapas secuenciales.
- `subsample=1.0` — Fracción de filas usada por etapa; un valor menor añade aleatoriedad y puede regularizar.
- `criterion` — Criterio para evaluar divisiones; según versión su default puede estar obsoleto, por lo que se suele dejar sin especificar.
- `min_samples_split=2` — Mínimo de filas para dividir; admite entero o fracción.
- `min_samples_leaf=1` — Mínimo de filas en cada hoja; admite entero o fracción.
- `min_weight_fraction_leaf=0` — Fracción mínima del peso total que debe llegar a una hoja.
- `max_depth=3` — Profundidad del árbol base; menor profundidad reduce complejidad.
- `min_impurity_decrease=0` — Reducción mínima de impureza para permitir una división.
- `init=None` — Inicialización por defecto; acepta un estimador inicial compatible.
- `random_state=None` — Controla subsampling y permutaciones; un entero reproduce el ajuste.
- `max_features=None` — Usa todas las variables; admite entero, fracción, `'sqrt'` o `'log2'`.
- `alpha=0.9` — Cuantil usado por pérdidas Huber y quantile.
- `verbose=0` — Nivel de mensajes de ajuste.
- `max_leaf_nodes=None` — Sin límite de hojas; un entero limita la complejidad.
- `warm_start=False` — Reajusta desde cero; verdadero conserva y agrega etapas al incrementar estimadores.
- `validation_fraction=0.1` — Fracción interna de entrenamiento reservada para parada temprana.
- `n_iter_no_change=None` — Deshabilita parada temprana; entero positivo define cuántas etapas sin mejora esperar.
- `tol=1e-4` — Mejora mínima para considerar que la pérdida sigue mejorando.
- `ccp_alpha=0` — Desactiva poda; valores mayores podan los árboles base.

La configuración del proyecto utiliza como referencia 100 etapas, tasa 0.05, profundidad 2 y semilla fija, y permite seleccionar pérdida y subsampling conforme a la firma del builder. No confundir los parámetros internos del árbol base con los del conjunto secuencial.

9.8. Particiones, validación y búsqueda

`train_test_split.`

- **`*arrays`** — Matrices/vectores con igual número de filas que se particionan conjuntamente.
- **`test_size=None`** — Entero (filas) o decimal entre 0 y 1 (proporción); si ambos tamaños son None, se reserva normalmente el 25 % para prueba.
- **`train_size=None`** — Entero o decimal entre 0 y 1; si se omite, se calcula como complemento de test.
- **`random_state=None`** — Sin semilla fija; un entero permite reproducir la partición.
- **`shuffle=True`** — Mezcla antes de separar; falso conserva el orden y exige `stratify=None`.
- **`stratify=None`** — Sin estratificación; una matriz de etiquetas mantiene proporciones de grupos, pensado principalmente para categorías discretas.

En series temporales no se debe barajar.

`KFold.`

- **`n_splits=5`** — Número de pliegues; entero al menos 2 y no mayor que el número de filas. Más pliegues aumentan cómputo y tamaño del entrenamiento por iteración.
- **`shuffle=False`** — Conserva el orden antes de asignar pliegues; verdadero mezcla las filas.
- **`random_state=None`** — Solo controla la mezcla cuando `shuffle=True`; un entero hace reproducible el reparto.

KFold no respeta grupos ni orden temporal; en esos casos se necesita otro splitter.

`RandomizedSearchCV.`

- **`estimator`** — Estimador compatible con la API de scikit-learn.
- **`param_distributions`** — Diccionario o lista de diccionarios que asigna nombres de parámetros (también anidados con `__`) a listas o distribuciones con método `rvs`. Con listas se muestrea sin reemplazo; con distribuciones, con reemplazo.
- **`n_iter=10`** — Candidatos que se prueban; elevarlo aumenta cobertura y coste.
- **`scoring=None`** — Usa la puntuación del estimador; también admite nombre, callable o varias métricas. En regresión `neg_mean_squared_error` se maximiza; más cerca de cero significa menor MSE.
- **`n_jobs=None`** — Ejecución en serie por defecto; `-1` solicita todos los núcleos.
- **`refit=True`** — Reajusta al ganador sobre todos los datos entregados; también admite nombre de métrica, callable o falso.
- **`cv=None`** — Usa cinco pliegues en versiones actuales; acepta entero, splitter o iterable de particiones.
- **`verbose=0`** — Silencio; valores mayores muestran detalles de los ajustes.
- **`pre_dispatch='2*n_jobs'`** — Limita tareas enviadas por anticipado y ayuda a controlar memoria.

- **random_state=None** — Sin semilla fija; un entero reproduce la selección aleatoria.
- **error_score=np.nan** — Valor asignado a candidatos que fallan; **'raise'** propaga el error.
- **return_train_score=False** — No guarda puntuaciones de entrenamiento; verdadero ayuda a diagnosticar sobreajuste a costa de más cómputo y memoria.

La búsqueda polinómica del proyecto fija normalmente 20 candidatos, cinco pliegues, semilla 42, MSE negativo y reajuste.

GridSearchCV.

- **estimator** — Estimador compatible con scikit-learn.
- **param_grid** — Diccionario de parámetros y secuencias de opciones, o lista de diccionarios para espacios condicionales; se recorren todas las combinaciones.
- **scoring** — Mismo criterio de puntuación descrito para `RandomizedSearchCV`.
- **n_jobs** — Número de trabajos paralelos; None usa uno y `-1` solicita todos los núcleos.
- **refit** — Reajusta el mejor candidato; admite booleano, métrica o callable.
- **cv** — Número de pliegues, splitter o iterable; None usa el default actual de cinco.
- **verbose** — Nivel de mensajes de progreso.
- **pre_dispatch** — Número de trabajos adelantados; reducirlo puede contener uso de memoria.
- **error_score=np.nan** — Puntuación para combinaciones fallidas; **'raise'** propaga el error.
- **return_train_score=False** — No guarda puntuaciones de entrenamiento salvo que se active.

El número de ajustes es candidatos por pliegues, más un reajuste si `refit` está activo. Este método se documenta como alternativa; no se usa en el código descrito.

9.9. Métricas e interpretación de scores

mean_absolute_error y mean_squared_error.

- **y_true** — Valores objetivo observados.
- **y_pred** — Valores predichos con forma compatible.
- **sample_weight=None** — Peso opcional por fila.
- **multioutput='uniform_average'** — Agrega objetivos con media uniforme; admite pesos explícitos o **'raw_values'** para devolver una métrica por objetivo.

MAE promedia errores absolutos; MSE eleva los errores al cuadrado y penaliza más los grandes. RMSE es la raíz cuadrada de MSE.

`r2_score`.

- `y_true` — Valores objetivo observados.
- `y_pred` — Valores predichos con forma compatible.
- `sample_weight=None` — Peso opcional por fila.
- `multioutput='uniform_average'` — Agrega objetivos con media uniforme; acepta pesos o `'raw_values'`.
- `force_finite=True` — Sustituye NaN/inf en el caso de objetivo constante por valores finitos; falso conserva esos resultados no finitos.

R^2 tiene máximo 1, una referencia de media igual a 0 y puede ser negativo. La disponibilidad de parámetros puede cambiar según la versión de sklearn.

9.10. Parámetros del wrapper, datos y valores de referencia

Los parámetros de aplicación como `test_size`, columnas de objetivo, rutas de CSV, identificadores de ejecución y configuración de Ollama son propios del proyecto, no del estimador sklearn. En la regresión polinómica la búsqueda opera dentro de entrenamiento y reserva una prueba final; no se debe ajustar el espacio mirando repetidamente esa prueba. El texto de Ollama explica resultados ya calculados y no modifica coeficientes ni puntuaciones.

Catálogo y parámetros de búsqueda del proyecto. En el proyecto IFCD107, `regression/parameter_sear` centraliza las listas candidatas por algoritmo. El método `get_param_distributions_iterator(model_name)` devuelve una copia del diccionario y el endpoint `GET /api/v1/regresion/parametros` muestra cada opción en una lista de elementos. La misma fuente alimenta el ajuste.

- `search=false` — Ajusta directamente los parámetros enviados en la solicitud.
- `search=true` — Usa `RandomizedSearchCV` con las listas centralizadas en Random Forest, KNN, SVR, AdaBoost y Gradient Boosting.
- `n_iter=10` — Número de combinaciones candidatas muestreadas.
- `search_cv=5` — Número de pliegues internos para seleccionar parámetros.
- `parameters` — En la respuesta, contiene los valores que realmente se entrenaron (los ganadores si se buscó).
- `requested_parameters` — Conserva los valores solicitados por el cliente para facilitar la trazabilidad.
- `search` — Resume activación, presupuesto, mejor configuración y puntuación de validación.

Se calcula la puntuación con `neg_mean_squared_error` dentro de entrenamiento y se mide el ganador en la prueba reservada. KNN anida la selección en cada pliegue externo al generar predicciones OOF. Solo se publican aquí los modelos implementados; XGBoost, LightGBM y CatBoost necesitan integrar sus estimadores y bibliotecas.

Listing 2: La lista publicada también alimenta RandomizedSearchCV

```
space = PARAMETER_CATALOG.get_param_distributions_iterator("svm")
search = RandomizedSearchCV(
    estimator=svr_pipeline,
    param_distributions=space,
    n_iter=10,
```

```
    cv=5,
    scoring="neg_mean_squared_error",
    refit=True,
)
search.fit(X_train, y_train)
best_pipeline = search.best_estimator_
best_parameters = search.best_params_
```

La colección Postman contiene la consulta del catálogo y campos `search`, `n_iter` y `search_cv` en las solicitudes de entrenamiento. El modo de búsqueda es explícito porque cada candidato requiere varios ajustes y el coste crece aproximadamente como candidatos por pliegues.

Cuadro 3: Valores de referencia citados en los builders del proyecto. No son valores universales óptimos.

Método	Valores de referencia	Lectura práctica
Lineal	test 20 %, semilla 42	Punto de comparación; confirmar que entrada admita el formato esperado.
KNN	$k = 5$, estandarización	Validar k con pliegues y comprobar k menor que cada partición de entrenamiento.
Polinómica	grados 1–5; 20 candidatos; 5 pliegues	Controla flexibilidad y coste; valorar estabilidad y error fuera de muestra.
Bosque aleatorio	200 árboles, semilla 42, profundidad libre	Aumenta estabilidad por promedio; limitar profundidad/hojas si hay sobreajuste o coste alto.
SVR	RBF, $C = 1$, $\epsilon = 0.1$, $\gamma =$ scale	Escalar predictores; ajustar C, ϵ y γ conjuntamente.
AdaBoost	100 etapas, tasa 0.05, árbol base poco profundo	Más etapas y tasa baja pueden equilibrarse; vigilar rendimiento de validación.
Gradient Boosting	100 etapas, tasa 0.05, profundidad 2	Ajustar conjuntamente tasa, etapas y complejidad del árbol.

10. Conclusiones

La organización modular permite estudiar cada algoritmo sin mezclarlo con el transporte HTTP. Los pipelines encapsulan preprocesamiento y estimador para evitar fugas durante la validación y facilitar la inferencia. La validación cruzada ayuda a seleccionar configuraciones; el conjunto de prueba ofrece una comprobación final; las métricas describen aspectos distintos del error. Las búsquedas aleatorias y exhaustivas son estrategias de cómputo, no garantías de calidad. En la variante polinómica, Ollama añade una explicación textual posterior, mientras que la evidencia cuantitativa sigue procediendo de predicciones y métricas calculadas por el modelo.

Referencias

[1] Scikit-learn, *Pipelines and composite estimators*. https://scikit-learn.org/stable/modules/compose.html

[2] Scikit-learn, *Cross-validation: evaluating estimator performance*. https://scikit-learn.org/stable/modules/cross_validation.html

[3] Scikit-learn, *RandomizedSearchCV*. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.RandomizedSearchCV.html

[4] Scikit-learn, *GridSearchCV*. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html

[5] Scikit-learn, *Model evaluation: quantifying the quality of predictions*. https://scikit-learn.org/stable/modules/model_evaluation.html

[6] NumPy, *numpy.polyfit*. https://numpy.org/doc/stable/reference/generated/numpy.polyfit.html

[7] Scikit-learn, documentación de `LinearRegression`. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html

[8] Scikit-learn, documentación de `KNeighborsRegressor`. https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsRegressor.html

[9] Scikit-learn, documentación de `RandomForestRegressor`. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html

[10] Scikit-learn, documentación de `SVR`. https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVR.html

[11] Scikit-learn, documentación de `AdaBoostRegressor`. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.AdaBoostRegressor.html

[12] Scikit-learn, documentación de `GradientBoostingRegressor`. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingRegressor.html

[13] Scikit-learn, documentación de imputación, codificación y escalado. https://scikit-learn.org/stable/modules/preprocessing.html

[14] Scikit-learn, documentación de `train_test_split` y validadores cruzados. https://scikit-learn.org/stable/modules/cross_validation.html

[15] Scikit-learn, documentación de `ColumnTransformer`. https://scikit-learn.org/stable/modules/generated/sklearn.compose.ColumnTransformer.html

[16] Scikit-learn, documentación de `Pipeline`. https://scikit-learn.org/stable/modules/generated/sklearn.pipeline.Pipeline.html